SOFTWARE TESTING

Skill Bootcamps — 3-Hour Comprehensive Lecture

Quality • Reliability • Confidence

HOUR 1

Foundations of Software Testing

What it is • Why it matters • Principles • SDLC & STLC • Levels

FOUNDATIONS

What Is Software Testing?

Definition

Software testing is the process of evaluating and verifying that a software application does what it is supposed to do. It involves executing a program with the intent of finding errors, gaps, or missing requirements.

Core Goals

- Detect bugs before release
- Validate requirements are met
- Ensure software quality & reliability
- Build confidence in the product
- Reduce maintenance costs

"The purpose of testing is to find problems early, when they're cheap to fix."

— Industry Principle

"Quality is not an act, it's a habit."

— Aristotle

FOUNDATIONS

Why Does Testing Matter?

Customer Satisfaction

Ensures the application meets user needs and expectations — building trust and loyalty.

Cost Savings

Fixing a bug in development costs 10× less than fixing it in production.

Quality Product

Identifies bugs, compatibility issues, and performance gaps before release.

Security

Uncovers vulnerabilities that could be exploited to breach sensitive data.

Reliability

Confirms the software performs consistently across environments and workloads.

Faster Development

Early defect detection reduces rework and speeds up delivery cycles.

PRINCIPLES

7 Principles of Software Testing

1 **1. Testing shows presence of defects**	Testing can reveal bugs but cannot prove a program is completely bug-free.
2 **2. Exhaustive testing is impossible**	Testing every possible input/condition is infeasible — risk-based prioritisation is key.
3 **3. Early testing saves cost**	Detecting defects early (shift-left) is far cheaper than late-stage fixes.
4 **4. Defect clustering**	A small number of modules contain most defects — focus effort on high-risk areas.
5 **5. Pesticide paradox**	Repeating the same tests stops finding new bugs — tests must be reviewed & updated.
6 **6. Testing is context-dependent**	Testing a banking app differs from testing a game — approach must match context.
7 **7. Absence-of-errors fallacy**	A bug-free system that fails user needs is still a failing system.

SDLC & STLC

SDLC vs STLC — How Testing Fits In

SDLC Phases

- Requirements Gathering
- System Design
- Coding / Development
- **Testing**
- Deployment
- Maintenance

STLC Phases

- Requirement Analysis
- Test Planning
- Test Case Design
- Test Environment Setup
- Test Execution
- Test Closure

TEST LEVELS

4 Levels of Software Testing

Unit Testing

Tests individual functions/components in isolation. Done by developers.
Tools: JUnit, NUnit, xUnit

Integration Testing

Tests how modules interact. Approaches: Top-Down, Bottom-Up, Big-Bang.
Focuses on interfaces and data flow.

System Testing

Tests the complete integrated system against requirements.
Includes functional & non-functional testing.

Acceptance Testing

Validates the system meets business needs (UAT). Done by end-users or QA.
Alpha & Beta testing fall here.

▲ *Increasing Complexity & Cost to Fix*

HOUR 2

Types of Software Testing

Manual vs Automation • White / Black / Gray box • Functional & Non-Functional

TYPES OF TESTING

Manual Testing vs Automation Testing

MANUAL TESTING

- Executed by human testers
- Suited for exploratory & UX testing
- Best for ad-hoc & unplanned changes
- No upfront tool investment
- Detects visual/layout issues easily
- Slower for repetitive tests

AUTOMATION TESTING

- Scripts run without manual effort
- Ideal for regression & smoke testing
- Higher initial cost; saves time long-term
- Increases coverage & repeatability
- Tools: Selenium, Cypress, JUnit
- Reduces human error significantly

TESTING APPROACHES

White Box · Black Box · Gray Box Testing

White Box Testing

Internal / Structural

- Tester has full access to source code
- Tests internal logic, paths and conditions
- Techniques: path testing, loop testing, mutation testing
- Used in unit & integration testing
- Finds hidden errors & unreachable code

Black Box Testing

Functional / Behavioural

- No knowledge of internal code required
- Tests based on requirements & specifications
- Techniques: equivalence partitioning, boundary value analysis
- Used in system & acceptance testing
- Simulates real user perspective

Gray Box Testing

Hybrid Approach

- Partial knowledge of internal structure
- Combines advantages of both approaches
- Techniques: matrix, pattern, orthogonal array testing
- Tests from user + developer perspective
- Improves overall product quality

FUNCTIONAL TESTING

Types of Functional Testing

Unit Testing

Tests smallest individual components. Developers write unit tests.

Integration Testing

Tests interaction between modules. Ensures combined components work.

System Testing

End-to-end test of complete system vs requirements.

User Acceptance Testing

Business stakeholders validate the system meets their needs (UAT).

Regression Testing

Re-tests after changes to ensure nothing is broken.

Smoke Testing

Quick sanity check — is the build stable enough for further testing?

Sanity Testing

Focused regression check on specific fixed bugs or features.

End-to-End Testing

Tests full user journeys across all system layers and integrations.

NON-FUNCTIONAL TESTING

Types of Non-Functional Testing

Performance Testing

Evaluates speed, responsiveness, and stability under load.

Load Testing

Tests system behaviour under expected concurrent users.

Stress Testing

Pushes system beyond limits to find breaking points.

Spike Testing

Sudden large bursts of traffic — can the system cope?

Endurance / Soak Testing

Sustained load over long periods — memory leaks, degradation.

Security Testing

Identifies vulnerabilities, threats, and security gaps.

Usability Testing

Evaluates ease-of-use and user experience.

Compatibility Testing

Tests across browsers, OS, devices, and environments.

Scalability Testing

Can the system scale up/down with increased demand?

Volume Testing

Tests with large data volumes in the database.

Integration Testing Strategies

Big-Bang Integration

All modules combined & tested at once. Simple but hard to isolate defects.

Top-Down Integration

Starts with highest-level modules, using stubs for lower levels. Interface errors found early.

Bottom-Up Integration

Starts with lowest-level modules, using drivers. Good for object-oriented systems.

Sandwich / Mixed Integration

Combines top-down and bottom-up. Best for large projects with many sub-systems.

HOUR 3

Practice, Tools & Techniques

Writing test cases • Defect lifecycle • Testing techniques • Automation tools

TEST CASE DESIGN

How to Write Effective Test Cases

Key Components of a Test Case:

Test Case ID	Test Title / Objective	Pre-conditions	Test Steps
Expected Result	Actual Result	Pass / Fail Status	Priority & Severity

Example Test Case — Login Feature:

ID	Title	Pre-condition	Steps	Expected	Actual	Status	Priority
TC_001	Verify login with valid credentials	User is on login page	1. Enter valid username 2. Enter valid password 3. Click Login	Dashboard is displayed	–	Pending	High

- Test cases should be clear, concise, and repeatable
- Positive tests (valid input) AND negative tests (invalid input) are both essential
- Document pre-conditions and any dependencies

DEFECT MANAGEMENT

Bug / Defect Lifecycle

New

Bug reported for the first time

Assigned

Assigned to developer for investigation

Open

Developer starts working on the fix

Fixed

Developer has applied the fix

Retest

Tester verifies the fix works

Closed

Verified fixed — bug is closed

Rejected

Not a valid bug (by design or misunderstood)

Deferred

Valid bug, postponed to a later release

AUTOMATION TOOLS

Popular Software Testing Tools

Unit Testing

JUnit (Java) · NUnit (.NET) · pytest (Python) · Mocha (JS)

UI / Functional

Selenium · Cypress · Playwright · TestCafe · Appium (mobile)

API Testing

Postman · REST Assured · SoapUI · Insomnia

Performance

Apache JMeter · Gatling · k6 · LoadRunner

Security Testing

OWASP ZAP · Burp Suite · Nessus · Metasploit

Test Management

Jira · TestRail · Zephyr · Azure DevOps · Xray

CI/CD Integration

Jenkins · GitHub Actions · CircleCI · GitLab CI

Mobile Testing

Appium · XCTest · Espresso · BrowserStack · Sauce Labs

KEY CONCEPTS

QA vs QC · Verification vs Validation

Quality Assurance (QA)

Process-oriented. Prevents defects from occurring.
'Are we building it right?'
Includes reviews, audits, process improvement.

Quality Control (QC)

Product-oriented. Identifies defects in the final product.
'Did we build it right?'
Includes testing, inspections, walkthroughs.

Verification

Checks that the software meets specifications.
"Are we building the product correctly?"
Static — reviews, walkthroughs, inspections.

Validation

Checks that the software meets user needs.
"Are we building the right product?"
Dynamic — actual test execution.

TEST DOCUMENTATION

Test Documentation Essentials

Test Plan

Master document defining scope, objectives, approach, schedule, resources, and risks for the entire test effort.

Test Strategy

High-level document covering testing approach, environments, tools, and exit criteria across the project.

Test Case

Step-by-step instruction to verify a specific requirement. Includes steps, expected results, and pass/fail criteria.

Test Scenario

High-level description of what to test (the 'what'). Multiple test cases derive from one scenario.

Traceability Matrix (RTM)

Maps each requirement to corresponding test cases, ensuring full coverage.

Bug Report / Defect Log

Documents defects with ID, steps to reproduce, expected vs actual result, severity, and priority.

DEFECT MANAGEMENT

Severity vs Priority in Defect Tracking

Severity Levels

Critical	System crash / data loss — complete blocker
Major	Key feature broken, no workaround available
Minor	Feature works but not as specified
Trivial	Cosmetic issue, does not affect functionality

Priority Levels

P1 - Urgent	Must be fixed immediately — blocks all testing/release
P2 - High	Fix before next build — major impact on users
P3 - Medium	Fix in this release cycle if possible
P4 - Low	Nice to fix — schedule for future sprint

Key Insight: High severity ≠ High priority. A critical bug in a rarely-used feature may be lower priority than a cosmetic bug on the checkout page.

Key Takeaways

1. Software testing is essential — it prevents failures, reduces cost, and builds user trust.
2. Testing follows the SDLC — the earlier defects are found, the cheaper they are to fix.
3. Know your testing types: Unit → Integration → System → Acceptance (the 4 levels).
4. Functional testing validates WHAT the system does; Non-functional tests HOW WELL it does it.
5. White, Black, and Gray box testing each serve different purposes and team roles.
6. Writing good test cases (clear, repeatable, covering positive & negative paths) is a core skill.
7. Defects need a full lifecycle: New → Assigned → Fixed → Retest → Closed.
8. Modern testing is Agile, automated, and continuous — testing is everyone's responsibility.

Questions & Discussion

Thank you for attending — Software Testing Skill Bootcamp

Further Learning: ISTQB Foundation Level · Selenium · Cypress · OWASP